Get started with DDC

You can interact with DDC through code using our SDK or visually through the Developer Console.

Quick Overview

What you'll learn:

- Create a DDC account
- Store your first file
- Retrieve stored files
- Choose between SDK (code) or Developer Console (UI)

Time to complete: ~10 minutes

Prerequisites

For SDK Development

- Node.js 16+ or compatible runtime
- npm or yarn package manager
- Basic understanding of JavaScript/TypeScript

For Developer Console

- Modern web browser (Chrome, Firefox, Safari, Edge)
- DDC account (we'll create this in the next step)

Choose Your Path

🖥️ Developer Console (UI)

Best for:

- Quick testing and exploration
- Visual file management
- Non-developers or low-code scenarios
- Understanding DDC concepts

Jump to Console Guide

💻 SDK (Code)

Best for:

- Production applications
- Automated workflows
- Custom integrations
- Advanced features and control

Jump to SDK Guide

Using the Developer Console

Step 1: Create Your Account

1. Navigate to the DDC Developer Console
2. Click **Sign Up** or **Create Account**
3. Complete registration with your email
4. Verify your email address
5. Log in to access your dashboard

Step 2: Create a Bucket

Buckets are containers for organizing your files, similar to folders or S3 buckets.

1. From your dashboard, click **Create Bucket**
2. Enter a bucket name (e.g., `my-first-bucket`)
3. Choose bucket privacy:
    - **Private**: Only you can access (requires authentication)
    - **Public**: Anyone with the link can read

4. Click **Create**

Step 3: Upload a File

1. Select your newly created bucket
2. Click **Upload File** or drag and drop
3. Choose a file from your computer
4. Wait for upload confirmation
5. Note the **Content ID (CID)** - this is your file's unique identifier

Step 4: Retrieve Your File

Option A: Through Console

1. Navigate to your bucket
2. Click on your uploaded file
3. Use the **Download** or **Preview** button

Option B: Direct Link

```
https://cdn.testnet.cere.network/<bucketid>/<CID>
```

Replace `<CID>` with your file's Content ID.

Using the SDK

Step 1: Generate a new account

```
npx @cere-ddc-sdk/cli account --random
```

Output

```
New account
Mnemonic: gospel fee escape timber toilet crouch artist catal
og salt icon bulb ivory
```

```
Type: sr25519
Address: 6PxrvjkVJFrQs6Tqdeov1GcJkS5rpzuM3NYGhX8KQErv49V2
```

> ⚠️ Store the Mnemonic (also called “seed phrase”) securely. Without this, you can’t recover you wallet and lose everything inside! Anyone who has access to this phrase, has full control over your account. If you prefer an account that you can recover by email, please use the 💰 Cere Wallet Wiki @Anonymous which requires the JSON Authentication method (code example) .

Top-up the account with CERE tokens either by using the faucet or by sending some amount of CERE tokens from another account.

```
6PxrvjkVJFrQs6Tqdeov1GcJkS5rpzuM3NYGhX8KQErv49V2
```

Step 2: Install the DDC SDK

```
npm install @cere-ddc-sdk/ddc-client --save
```

Or with yarn:

```
yarn add @cere-ddc-sdk/ddc-client
```

Step 3: Initialize the Client

```
import { DdcClient, TESTNET } from '@cere-ddc-sdk/ddc-clien
t';

const seedPhrase = 'gospel fee escape timber toilet crouch ar
tist catalog salt icon bulb ivory';
const client = await DdcClient.create(seedPhrase, TESTNET);
```

Step 4: Create a Bucket

Obtain the Cluster ID of the cluster you want to use. For testnet, use: 0x825c4b2352850de9986d9d28568db6f0c023a1e3

```
// Create a new bucket
const clusterId ='0x825c4b2352850de9986d9d28568db6f0c023a1e
3';
const bucketId = await client.createBucket(clusterId);

console.log('Bucket created with ID:', bucketId);
```

Step 5: Store a File

```
import * as fs from 'fs';
import { File } from '@cere-ddc-sdk/ddc-client';

const filePath = './my-picture.jpg';
const fileStats = fs.statSync(filePath);
const fileStream = fs.createReadStream(filePath);
const file = new File(fileStream, { size: fileStats.size });

const fileUri = await client.store(bucketId, file);

console.log('File stored into bucket', bucketId, 'with CID',
fileUri.cid);
console.log('The file can be accessed by this URL', `https://
storage.testnet.cere.network/${bucketId}/${fileUri.cid}`);
```

Step 6: Retrieve a File

```
import * as fs from 'fs';
import { Readable } from 'stream';
import { pipeline } from 'stream/promises';
import { FileUri } from '@cere-ddc-sdk/ddc-client';
```

```
const outputFilePath = './my-downloaded-picture.jpg';

const uri = new FileUri(bucketId, fileCid);
const fileResponse = await client.read(uri);
const outputFileStream = fs.createWriteStream(outputFilePat
h);

await pipeline(Readable.fromWeb(fileResponse.body), outputFil
eStream);

console.log('The file downloaded to', outputFilePath);
```

Complete Example

Here's a full working example combining all steps:

```
import path from 'path';
import fs from 'fs';
import { fileURLToPath } from 'url';
import { Readable } from 'stream';
import { pipeline } from 'stream/promises';
import { DdcClient, File, TESTNET, AuthToken, UriSigner, Auth
TokenOperation } from '@cere-ddc-sdk/ddc-client';

const CERE = 10_000_000_000n;

// The wallet should have enough CERE to pay for the transact
ion fees
const user = 'gospel fee escape timber toilet crouch artist c
atalog salt icon bulb ivory';
const userSigner = new UriSigner(user);
const clusterId = '0x825c4b2352850de9986d9d28568db6f0c023a1e
3';
const dir = path.dirname(fileURLToPath(import.meta.url));
const inputFilePath = path.resolve(dir, 'example.jpg');
```

```
const outputFilePath = path.resolve(dir, 'example-downloaded.
jpg');

// Create a DDC client instance and connect it to DDC TESTNET
const client = await DdcClient.create(user, TESTNET);

//Get current deposit of the DDC customer to see if there are
enough funds
const deposit = await client.getDeposit(clusterId);
console.log('Current deposit', deposit / CERE, 'CERE');

if (deposit === 0n) {
  await client.depositBalance(clusterId, 5n * CERE);
  console.log('Deposited 5 CERE');
}

// Create a public bucket
const bucketId = await client.createBucket(clusterId, {
  isPublic: true,
});

console.log('Public bucket created', bucketId);

const inputFileStats = fs.statSync(inputFilePath);
const inputFileStream = fs.createReadStream(inputFilePath);
//Create a DDC file instance
const ddcFile = new File(inputFileStream, {
  size: inputFileStats.size,
});

// Store the file into DDC
const fileUri = await client.store(bucketId, ddcFile);
console.log('File stored into bucket', bucketId, 'with CID',
fileUri.cid);
console.log('The file can be accessed by this URL', `https://
cdn.testnet.cere.network/${bucketId}/${fileUri.cid}`);
```

```
// For private buckets, generate an access token
const accessToken = await client.grantAccess(userSigner.addre
ss, {
  bucketId,
  operations: [AuthTokenOperation.GET],
  pieceCid: fileUri.cid,
});

const fileUrl = `https://cdn.testnet.cere.network/${bucketI
d}/${fileUri.cid}?token=${accessToken}`;
console.log('The file URL with access token', fileUrl);

// Read the file from DDC
const fileResponse = await client.read(fileUri, { accessToken
});
const outputFileStream = fs.createWriteStream(outputFilePat
h);

// Stream the content to the destination file
await pipeline(Readable.fromWeb(fileResponse.body), outputFil
eStream);
console.log('The file downloaded to', path.basename(outputFil
ePath));

// Disconnect the client
await client.disconnect();
```

Next Steps with SDK

- Explore advanced bucket operations
- Implement file metadata and tags
- Set up content encryption (client-side)
- Handle errors and retry logic

Understanding DDC Architecture

Now that you've stored and retrieved your first file, learn more about how DDC works:

- **Architecture Overview** - Understand DDC's decentralized storage model
- **Data Security** - Learn about encryption, access control, and node operators
- **Erasure Coding** - How DDC distributes data across nodes
- **Buckets & Permissions** - Deep dive into access control and bucket management

---

Code Examples & Resources

Gitbook

- Official Gitbook

GitHub Repositories

Official Examples:

- DDC SDK Examples - Comprehensive code samples
- CLI Examples - CLI Examples
- Node.js Examples- Server-side integration